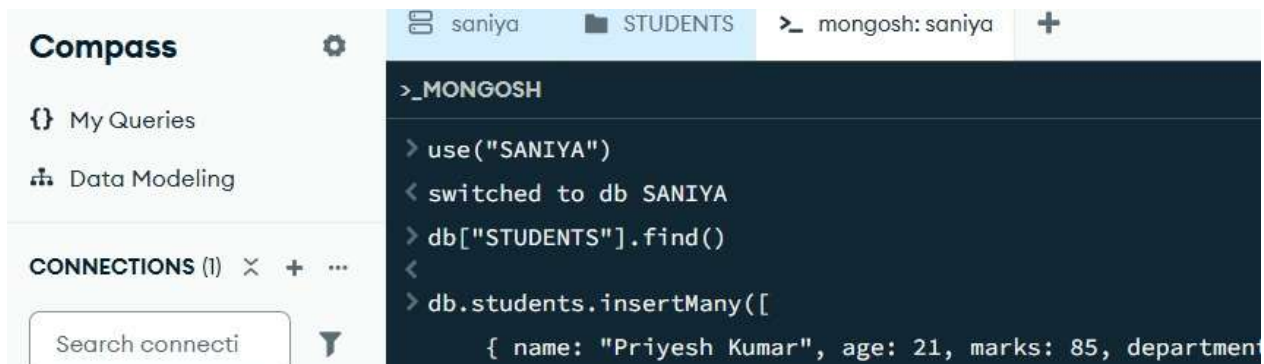


1. CREATE DATABASE

MongoDB Query:

use Students

Explanation: Creates or switches to the Student database.

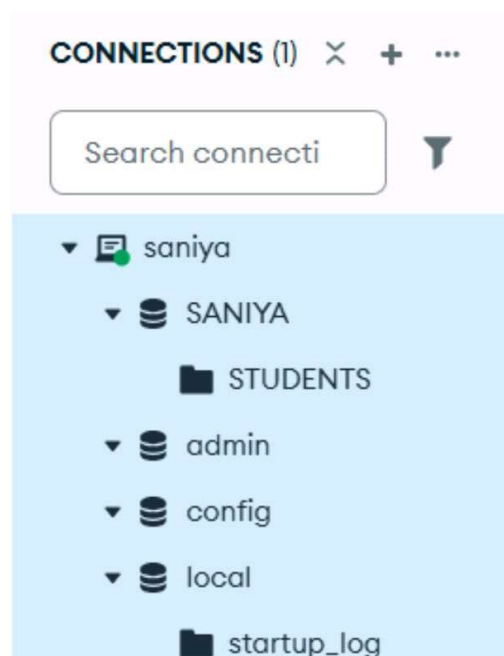


2. CREATE COLLECTION

MongoDB Query:

db.create Collection("SANIYA")

Explanation: Creates a collection named students.



3. INSERT 30 STUDENTS

MongoDB Query:

```
db.students.insertMany([  
    { name: "Priyesh Kumar", age: 21, marks: 85, department: "CSE", skills: ["Python",  
"MongoDB", "Java"] },  
    { name: "Saniya Khan", age: 22, marks: 72, department: "ECE", skills: ["Python",  
"SQL"] },  
    { name: "Prishi", age: 20, marks: 91, department: "CSE", skills: ["Java", "MongoDB",  
"AI"] },  
    { name: "Rohit", age: 23, marks: 65, department: "ME", skills: ["AutoCAD",  
"Python"] },  
    { name: "Sanu", age: 21, marks: 78, department: "CSE", skills: ["Python", "AI"] },  
    { name: "Ranveer", age: 22, marks: 88, department: "ECE", skills: ["C", "Java",  
"SQL"] },  
    { name: "Rijul", age: 24, marks: 69, department: "ME", skills: ["AutoCAD",  
"SolidWorks"] },  
    { name: "Rupanshu", age: 20, marks: 94, department: "CSE", skills: ["Python", "AI",  
"MongoDB"] },  
    { name: "Priyanshu", age: 23, marks: 81, department: "IT", skills: ["Java", "SQL",  
"HTML"] },  
    { name: "Sakshi", age: 21, marks: 76, department: "ECE", skills: ["Python", "C",  
"Embedded"] },  
    { name: "Arjun", age: 22, marks: 89, department: "CSE", skills: ["Java", "Spring",  
"MongoDB"] },  
    { name: "Kavita", age: 20, marks: 83, department: "IT", skills: ["Python", "HTML",  
"CSS"] },
```

```
{ name: "Manish", age: 24, marks: 61, department: "ME", skills: ["AutoCAD",  
"Python"] },  
  
{ name: "Riya", age: 21, marks: 95, department: "CSE", skills: ["Python", "AI", "ML"] },  
  
{ name: "Suresh", age: 23, marks: 74, department: "ECE", skills: ["C", "Java",  
"Arduino"] },  
  
{ name: "Simran", age: 22, marks: 87, department: "IT", skills: ["JavaScript", "React",  
"Node.js"] },  
  
{ name: "Deepak", age: 21, marks: 70, department: "ME", skills: ["AutoCAD",  
"MATLAB"] },  
  
{ name: "Nisha", age: 20, marks: 92, department: "CSE", skills: ["Python",  
"MongoDB", "Django"] },  
  
{ name: "Varun", age: 24, marks: 79, department: "IT", skills: ["Java", "SQL", "Spring"]  
},  
  
{ name: "Meena", age: 22, marks: 86, department: "ECE", skills: ["Python", "VLSI",  
"C"] },  
  
{ name: "Aditya", age: 21, marks: 90, department: "CSE", skills: ["Python", "Java",  
"AI"] },  
  
{ name: "Shreya", age: 23, marks: 68, department: "ME", skills: ["AutoCAD",  
"CATIA"] },  
  
{ name: "Akash", age: 22, marks: 77, department: "IT", skills: ["HTML", "CSS",  
"JavaScript"] },  
  
{ name: "Divya", age: 20, marks: 93, department: "CSE", skills: ["Python", "ML",  
"TensorFlow"] },  
  
{ name: "Nitin", age: 24, marks: 64, department: "ECE", skills: ["C", "Embedded",  
"Arduino"] },  
  
{ name: "Swati", age: 21, marks: 82, department: "IT", skills: ["Python", "SQL",  
"PowerBI"] },
```

```

    { name: "Mohit", age: 23, marks: 75, department: "CSE", skills: ["Java", "SQL",
"MongoDB"] },

    { name: "Komal", age: 20, marks: 89, department: "ECE", skills: ["Python", "Java",
"IoT"] },

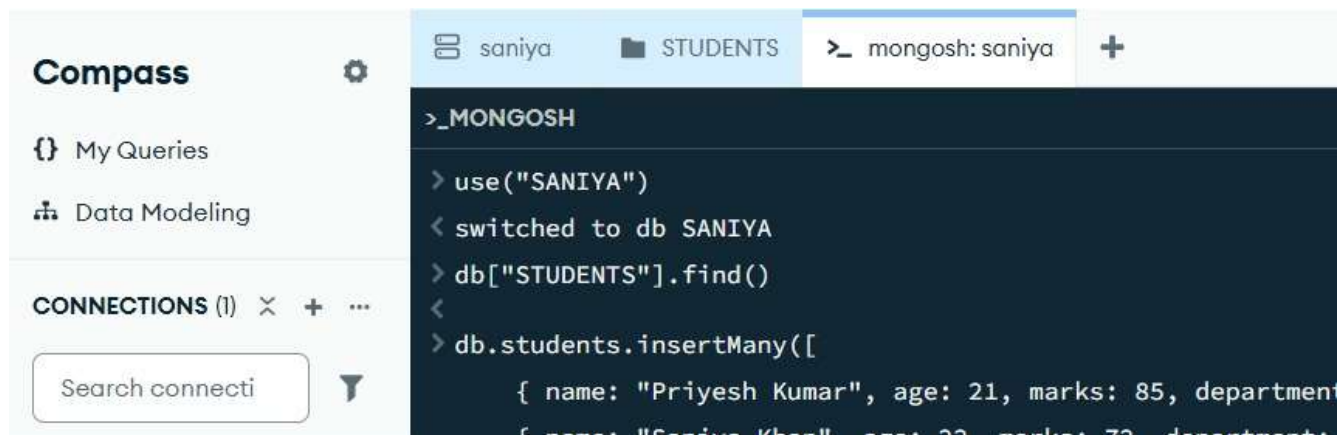
    { name: "Yash", age: 22, marks: 71, department: "ME", skills: ["AutoCAD", "MATLAB",
"SolidWorks"] },

    { name: "Isha", age: 21, marks: 96, department: "CSE", skills: ["Python", "AI", "Deep
Learning"] }

]);

```

Explanation: Inserts multiple student documents into the collection.



4. DISPLAY ALL STUDENTS

MongoDB Query:

```
db.students.find()
```

Explanation: Retrieves documents from the students collection.

```
> db.students.find()
< {
  _id: ObjectId('6ab94b3502beebe9309cf74e'),
  name: 'Priyesh Kumar',
  age: 21,
  marks: 85,
  department: 'CSE',
  skills: [
    'Python',
    'MongoDB',
    'Java'
  ]
}
{
  _id: ObjectId('6ab94b3502beebe9309cf74f'),
  name: 'Saniya Khan',
  age: 22,
  marks: 72,
  department: 'ECE',
  skills: [
    'Python',
    'SQL'
  ]
}
{
  _id: ObjectId('6ab94b3502beebe9309cf750'),
  name: 'Prishu',
  age: 20,
```

5. EQUAL TO — \$eq

MongoDB Query:

db. students. Find ({marks: {\$eq: 88}})

Explanation: Finds students whose marks are exactly 88.

```
> db.students.find({marks: {$eq: 88}})
```

```
< {  
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),  
  roll: 1,  
  name: 'Anamika',  
  age: 19,  
  marks: 88,  
  city: 'Lucknow'  
}  
  
{  
  _id: ObjectId('6aae9ea78f0f98eb52df2236'),  
  roll: 27,  
  name: 'Ananya',  
  age: 19,  
  marks: 88,  
  city: 'Varanasi'  
}
```

6. GREATER THAN — \$gt

MongoDB Query:

```
db.students.find({marks: {$gt: 72}})
```

Explanation: Finds students whose marks are greater than 72.

7.

```
> db.students.findOne({marks : {$eq : 72}})
< {
  _id: ObjectId('6ab94b3502beebe9309cf74f'),
  name: 'Saniya Khan',
  age: 22,
  marks: 72,
  department: 'ECE',
  skills: [
    'Python',
    'SQL'
  ]
}
```

SANIYA > |

LESS

THAN — \$lt

MongoDB Query:

```
db. students. find ({marks: {$lt: 72}})
```

Explanation: Finds students whose marks are less than 72.

```
> db.students.findOne({marks : {$lt : 72}})
```

```
< {  
  _id: ObjectId('6ab94b3502beebe9309cf751'),  
  name: 'Rohit',  
  age: 23,  
  marks: 65,  
  department: 'ME',  
  < skills: [  
    'AutoCAD',  
    'Python'  
  ]  
}
```

```
SANIYA >
```

8. GREATER THAN OR EQUAL TO — \$gte

MongoDB Query:

db.students.find({marks: {\$gte: 72}})

Explanation: Finds students whose marks are 72 or above.

```
> db.students.findOne({marks : {$gt : 72}})
< {
  _id: ObjectId('6ab94b3502beebe9309cf74e'),
  name: 'Priyesh Kumar',
  age: 21,
  marks: 85,
  department: 'CSE',
  skills: [
    'Python',
    'MongoDB',
    'Java'
  ]
}
```

SANIYA >

9. LESS THAN OR EQUAL TO — \$lte

MongoDB Query:

db.students.find({marks: {\$lte: 88}})

```
> db.students.findOne({marks : {$lte : 88}})
< {
  _id: ObjectId('6ab94b3502beebe9309cf74e'),
  name: 'Priyesh Kumar',
  age: 21,
  marks: 85,
  department: 'CSE',
  skills: [
    'Python',
    'MongoDB',
    'Java'
  ]
}
```

SANIYA > |

Explanation: Finds students whose marks are 88 or below.

10. UPDATE OPERATION

MongoDB Query:

```
db.students.updateOne(  
  {age:21},  
  {$set:{marks:50}}  
)
```

Explanation: Updates the marks to 50 of the student with age 21.

```
> db.students.updateOne(  
  {age : 21},  
  {$set:{marks:50}}  
)  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

11. DELETE ONE OPERATION

MongoDB Query:

```
> db.students.deleteOne({age : 21})
< {
  acknowledged: true,
  deletedCount: 1
}
SANIYA >
```

db.students.deleteOne({age:21})

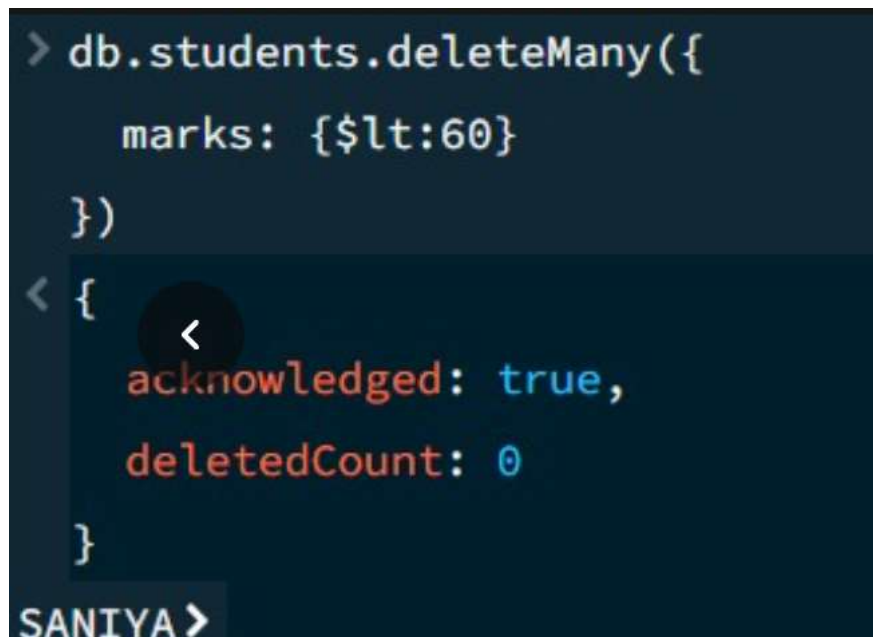
Explanation: Deletes one document matching age 21.

12. DELETE MANY OPERATION

MongoDB Query:

```
db.students.deleteMany({  
  marks: {$lt:60}  
})
```

Explanation: Deletes all students having marks below 60.



```
> db.students.deleteMany({  
  marks: {$lt:60}  
  })  
< {  
  acknowledged: true,  
  deletedCount: 0  
}
```

SANIYA >

13. AND OPERATION — \$and

MongoDB Query:

```
db.students.find({
  $and:[
    {marks:{>40}},
    {age:22}
  ]
})
```

Explanation: Finds students satisfying both conditions.

```
> db.students.findOne({$and:[{marks : {> 40}},{age : 22}]})
< {
  _id: ObjectId('6ab94b3502beebe9309cf74f'),
  name: 'Saniya Khan',
  age: 22,
  marks: 72,
  department: 'ECE',
  skills: [
    'Python',
    'SQL'
  ]
}
```

SANIYA > |

14. OR OPERATION — \$or

MongoDB Query:

```
db.students.find({
  $or:[
    {name:"Saniya Khan"},
    {name:"Prishi"}
  ]
})
```

Explanation: Finds students name Saniya Khan or Prishi.

```
> db.students.find({$or : [{name : "Saniya Khan"},{name : "Prishi"}]})
< {
  _id: ObjectId('6ab94b3502beebe9309cf74f'),
  name: 'Saniya Khan',
  age: 22,
  marks: 72,
  department: 'ECE',
  skills: [
    'Python',
    'SQL'
  ]
}
{
  _id: ObjectId('6ab94b3502beebe9309cf750'),
  name: 'Prishi',
  age: 20,
  marks: 91,
  department: 'CSE',
  skills: [
    'Java',
    'MongoDB',
    'AI'
  ]
}
SANIYA > |
```

15. SORT OPERATION

A.MongoDB Query:

`db.students.find().sort({marks : 1})`

Explanation: Sorts students by marks from lowest to highest. Use 1 for ascending order.

```
> db.students.find().sort({marks : 1})
< {
  _id: ObjectId('6ab94b3502beebe9309cf75a'),
  name: 'Manish',
  age: 24,
  marks: 61,
  department: 'ME',
  skills: [
    'AutoCAD',
    'Python'
  ]
}
{
  _id: ObjectId('6ab94b3502beebe9309cf766'),
  name: 'Nitin',
  age: 24,
  marks: 64,
  department: 'ECE',
  skills: [
    'C',
    'Embedded',
    'Arduino'
  ]
}
{
  _id: ObjectId('6ab94b3502beebe9309cf751'),
  name: 'Rohit',
```

B. MongoDB Query:

db.students.find().sort({marks: -1})

Explanation: Sorts students by marks from highest to lowest. Use -1 for descending order.

```
> db.students.find().sort({marks : -1})
< {
  _id: ObjectId('6ab94b3502beebe9309cf76b'),
  name: 'Isha',
  age: 21,
  marks: 96,
  department: 'CSE',
  skills: [
    'Python',
    'AI',
    'Deep Learning'
  ]
}
{
  _id: ObjectId('6ab94b3502beebe9309cf75b'),
  name: 'Riya',
  age: 21,
  marks: 95,
  department: 'CSE',
  skills: [
    'Python',
    'AI',
    'ML'
  ]
}
{
  _id: ObjectId('6ab94b3502beebe9309cf755'),
```

16. LIMIT OPERATION

MongoDB Query:

db.students.find().limit(5)

Explanation: Displays only the first five documents.

```
> db.students.find().limit(5)
< {
  _id: ObjectId('6ab94b3502beebe9309cf74f'),
  name: 'Saniya Khan',
  age: 22,
  marks: 72,
  department: 'ECE',
  skills: [
    'Python',
    'SQL'
  ]
}
{
  _id: ObjectId('6ab94b3502beebe9309cf750'),
  name: 'Prishi',
  age: 20,
  marks: 91,
  department: 'CSE',
  skills: [
    'Java',
    'MongoDB',
    'AI'
  ]
}
{
  id: ObjectId('6ab94b3502beebe9309cf751'),
```

17. SEARCH BY CITY

MongoDB Query:

```
db.students.find({age:22})
```

Explanation: Finds all students whose age is 22.

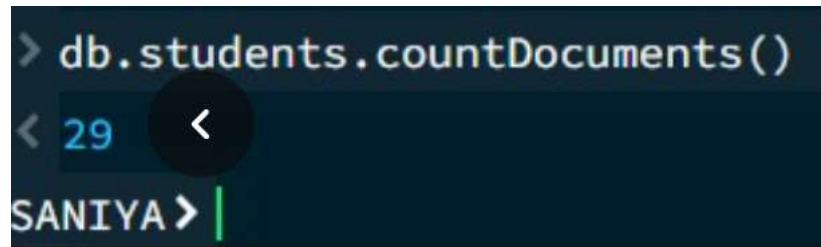
```
> db.students.find({age : 22})
< {
  _id: ObjectId('6ab94b3502beebe9309cf74f'),
  name: 'Saniya Khan',
  age: 22,
  marks: 72,
  department: 'ECE',
  skills: [
    'Python',
    'SQL'
  ]
}
{
  _id: ObjectId('6ab94b3502beebe9309cf753'),
  name: 'Ranveer',
  age: 22,
  marks: 88,
  department: 'ECE',
  skills: [
    'C',
    'Java',
    'SQL'
  ]
}
{
  _id: ObjectId('6ab94b3502beebe9309cf758'),
  name: 'Arjun'
```

18. COUNT DOCUMENTS

MongoDB Query:

db. students. count Documents ()

Explanation: Counts documents currently present in the collection (performing after the delete function).



```
> db.students.countDocuments()  
< 29  
SANIYA > |
```

A screenshot of a MongoDB command prompt window. The first line shows the command `> db.students.countDocuments()` being entered. The second line shows the result `< 29`. The prompt then changes to `SANIYA > |`.

MONGODB OPERATORS USED:

Operator	Meaning	Example
<code>\$eq</code>	Equal to	marks: <code>{ \$eq:88 }</code>
<code>\$gt</code>	Greater than	marks: <code>{ \$gt:72 }</code>
<code>\$lt</code>	Less than	marks: <code>{ \$lt:88 }</code>
<code>\$gte</code>	Greater than or equal to	marks: <code>{ \$gte:72 }</code>
<code>\$lte</code>	Less than or equal to	marks: <code>{ \$lte:88 }</code>
<code>\$and</code>	All conditions true	<code>\$and:[...]</code>
<code>\$or</code>	At least one condition true	<code>\$or:[...]</code>

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update, and delete documents. I also learned how to Ascending order & descending order data, comparison operators such as `$eq`, `$gt`, `$lt`, `$gte`, and `$lte` work, along with logical operators such as `$and` and `$or`.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.